Enterprise App Catalog Deployment in Microsoft Intune — Prepackaged Win32 Apps & Auto-Update

Implementation Solution Guide

Version 1.0.0 · 2026-07-21

Contents

Executive Summary — Enterprise App Catalog Deployment in Microsoft Intune (Prepackaged Win32 Apps & Auto-Update) 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 4

1. Solution Overview — Enterprise App Catalog Deployment in Microsoft Intune 5

What the Enterprise App Catalog is . . . 5

How it works . . . 5

Where it fits (three lanes for Windows apps) . . . 6

Benefits . . . 6

Limitations (be honest with buyers) . . . 6

The standard deployment model . . . 7

2. Requirements Guide — Enterprise App Catalog Deployment in Microsoft Intune 7

2.1 Licensing . . . 7

2.2 Device enrollment and the Intune Management Extension (IME) . . . 7

2.3 The app is present in the Enterprise App Catalog . . . 8

2.4 Install context decision (system vs user) . . . 8

2.5 Update management understood and decided . . . 8

2.6 Pilot and broad ring groups . . . 8

2.7 Network access and content delivery . . . 9

2.8 National cloud / Government tenant differences (if applicable) . . . 9

Checklist . . . 9

Global Requirements — Intune Solutions (category-level) 9

G1. Microsoft Entra ID tenant . . . 9

G2. Microsoft Intune subscription . . . 9

G3. Administrative roles . . . 10

G4. Device enrollment in Intune . . . 10

G5. Pilot security group . . . 10

G6. Supported Windows baseline . . . 10

G7. Network access to Microsoft cloud endpoints . . . 10

3. Implementation Guide — Enterprise App Catalog Deployment in Microsoft Intune 10

Named objects (reuse these names verbatim everywhere) . . . 11

Step 1 — Create the ring groups . . . 11

Step 2 — Start adding an Enterprise App Catalog app . . . 11

Step 3 — Select the app from the catalog . . . 11
Step 4 — Confirm install behavior (the supplied package settings) . . . 12
Step 5 — Verify the built-in detection rule . . . 12
Step 6 — Enable update management . . . 12
Step 7 — Assign to the pilot ring . . . 12
Step 8 — Apply and validate on a pilot device . . . 13
Step 9 — Expand to the broad ring . . . 13
5. Testing Guide — Enterprise App Catalog Deployment in Microsoft Intune 13
Test 1 — App added from the catalog . . . 14
Test 2 — App reports applicable to the pilot ring . . . 14
Test 3 — Install succeeds on the device . . . 14
Test 4 — Built-in detection agrees with Intune (reporting is true) . . . 14
Test 5 — Update management is enabled . . . 14
Test 6 — An update applies via update management . . . 14
Test 7 — Fleet report . . . 14
Pass/fail gate . . . 15
6. Troubleshooting Guide — Enterprise App Catalog Deployment in Microsoft Intune 15
A. The app I want isn't in the catalog . . . 15
B. App shows Not applicable . . . 15
C. Install fails outright . . . 16
D. Update isn't applying (app stays on the old version) . . . 16
E. Detection mismatch (installed but shows Failed, or vice-versa) . . . 17
F. Assignment scope / nothing deploys . . . 17
G. Government / national-cloud differences . . . 17
H. Diagnostics quick reference . . . 17
I. When to escalate . . . 17
7. Best Practices — Enterprise App Catalog Deployment in Microsoft Intune 18
8. Deliverables Summary — Enterprise App Catalog Deployment in Microsoft Intune 18
Documentation . . . 18
PowerShell scripts . . . 19
What the buyer receives . . . 19
Outcome . . . 19
Appendix A — PowerShell Scripts 21

Executive Summary — Enterprise App Catalog Deployment in Microsoft Intune (Prepackaged Win32 Apps & Auto-Update)

White-label · Intune Solutions

Hand-packaging every Windows application — wrapping the installer, writing install/uninstall commands, authoring a detection rule, then chasing each new version — is slow and never-ending. Microsoft's **Enterprise App Catalog** changes that: it is a **curated set of prepackaged Windows (Win32) apps** with install logic and detection **already built**, plus **update management** so deployed catalog apps stay current without re-wrapping. You pick the app from the catalog, choose how it installs and who gets it, turn on update management, and roll it out pilot-first.

This is a complete, **deploy-it-today** package for deploying applications from the Enterprise App Catalog with Microsoft Intune, written for an administrator with no prior packaging experience.

> ***Note — not Entra "Enterprise applications."*** *This solution covers the **Intune Enterprise App *Catalog** (prepackaged Windows apps you deploy to devices). It is **not** the Entra ID "Enterprise applications" blade (app registrations, service principals, API permissions) — that is identity/OAuth governance, a separate discipline and product.*

What you get

- A plain-language **Solution Overview** of the Enterprise App Catalog, the three Windows-app lanes (catalog vs hand-packaged Win32 vs winget/Store), built-in detection, and update management.
- A **Requirements Guide** with verify / implement-if-missing for every prerequisite.
- An **Implementation Guide** of precise, click-by-click Intune steps in the required order — add the catalog app, select it, confirm install behavior, verify the built-in detection, enable update management, assign a pilot ring, expand.
- Five **PowerShell scripts** for discovery, pass/fail validation, install/update reporting, on-device testing, and a `-WhatIf`-gated pilot assignment.
- **Testing** and **Troubleshooting** guides for confident rollout.

What this delivers

A repeatable catalog-deployment method — pick a **prepackaged** app, deploy it with Microsoft's **built-in detection**, keep it current with **update management**, and prove it on a **pilot ring** before scaling — with the exact-navigation specifics flagged to **verify against current Microsoft documentation** so the guide stays accurate as the portal evolves.

The deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand.** The pilot ring is the gate.

Outcome

Common Windows applications that deploy predictably from a curated source and stay current automatically — deployable by a single administrator and validated before fleet rollout.

1. Solution Overview — Enterprise App Catalog Deployment in Microsoft Intune

What the Enterprise App Catalog is

The **Enterprise App Catalog** in Microsoft Intune is a Microsoft-curated set of **prepackaged Windows (Win32) applications**. Each catalog app arrives with its **install logic and detection rule already built** by Microsoft, so you do not wrap the installer, write the install/uninstall command lines, or author a detection rule yourself. On top of that, catalog apps support **update management**, so a deployed catalog app can be kept current as new versions are published — without re-wrapping and re-uploading a fresh `.intunewin` each time.

In short: instead of hand-building a Win32 package for a common application, you **pick it from the catalog**, choose how it installs and who gets it, turn on update management, and roll it out pilot-first.

> ***Not the same as "Enterprise Applications" in Entra.*** *This solution is about the **Intune Enterprise App *Catalog** — curated, prepackaged Windows *apps you deploy to devices*. It is **not** the Entra ID **"Enterprise applications"** blade (app registrations, service principals, API permissions, SSO). Those govern *identity and OAuth access for SaaS/registered apps* — a completely different discipline covered by a separate governance product. If you came here for app registrations / service-principal governance, this is the wrong product.*

How it works

1. In the **Microsoft Intune admin center**, you add a new app and choose the **Enterprise App Catalog app** type (exact menu label — *verify against current Microsoft documentation*).
1. You **browse or search the catalog** and select the app (and, where offered, a specific version).

Which applications appear in the catalog, and which versions, is defined by Microsoft and changes over time — always confirm the app you want is present in your tenant's catalog (verify against current Microsoft documentation).

1. Microsoft's **prebuilt package** supplies the install/uninstall behavior and the **detection rule **— you do not author them. You confirm the** install behavior** (system vs user context) and other supplied settings.
1. You enable **update management** so the catalog app is kept current as Microsoft publishes new versions (the exact toggle name, cadence, and update behavior are *verify-with-Microsoft*).
1. You **assign** the app — **Required**, **Available** (via Company Portal), or **Uninstall** — targeted to a **pilot ring** first, then to the broad fleet.
1. The **Intune Management Extension (IME)** on the enrolled device downloads the content, installs

the app in the chosen context, evaluates the built-in detection rule, and reports **Installed / Failed / Not applicable** back to Intune. Update management then keeps it current on its cycle.

Where it fits (three lanes for Windows apps)

Intune gives you more than one way to get a Windows app onto a device. Choose deliberately:

Lane	Use it when	Who builds the package
Enterprise App Catalog (this product)	The app is a **common application present in the catalog** and you want Microsoft's prebuilt package + update management, no wrapping.	**Microsoft** (curated, prepackaged)
Hand-packaged Win32 (`.intunewin`)	The app is **not in the catalog**, or you need full control of install/uninstall/detection/requirements.	**You** (Win32 Content Prep Tool)
Winget / Microsoft Store apps	The app is available via **winget / the Microsoft Store** and you prefer that source.	**Microsoft Store / winget**

> *The **winget / Microsoft Store** lane is its own solution — see the sibling product **"Winget & Microsoft Store App Management with Microsoft Intune"** for that path. This product does **not** reproduce it; it points to it so you pick the right lane. When a catalog app fails to install or update, use the existing break/fix product **"Intune App Deployment Troubleshooting & Failure Remediation (Win32/LOB/Store)"** for deep diagnostics — the Troubleshooting section here covers the catalog-specific first steps and hands off there.*

Benefits

• **No hand-packaging** — Microsoft supplies the package, install logic, and detection for catalog apps, so you skip the Content Prep Tool and detection-rule authoring for those apps.

• **Stays current** — update management keeps deployed catalog apps up to date without re-wrapping.

• **Fewer false states** — Microsoft's built-in detection rule reduces the "installed but shows failed" and reinstall-loop problems that come from a hand-authored detection guess.

• **Consistent process** — the same **pilot ring** → **validate** → **expand** model as every other app deployment, so a bad rollout hits a pilot, never the whole fleet at once.

• **Faster to deploy common apps** — pick, assign, done — for the apps that are in the catalog.

Limitations (be honest with buyers)

• **Only apps in the catalog qualify.** If the app you need is not curated by Microsoft, you must use hand-packaged Win32 or the winget/Store lane instead. *The catalog's contents are defined by Microsoft and change — verify availability.*

• **You don't control the package internals.** Install command, detection, and packaging are

Microsoft's; for apps needing custom install behavior, hand-packaged Win32 is the right lane.

• **Update-management behavior is Microsoft-defined.** Cadence, version pinning, and how updates are

applied are set by the service — *verify the current behavior and options against Microsoft docs* before you rely on a specific update guarantee.

• **The IME must be present and healthy** on the enrolled Windows device (it installs automatically

when a Win32/catalog app or script is assigned).

• **Licensing and feature availability may differ by cloud** (Commercial vs GCC / GCC High / DoD) —

verify for your tenant and national cloud.

The standard deployment model

> ***Assess*** *(confirm the app is in the catalog and pick it)* → ***Configure*** *(install behavior + assignment, enable update management)* → ***Pilot*** *(assign to a pilot ring)* → ***Validate*** *(install, detection, and an update all succeed)* → ***Expand*** *(wave to the broad ring). The pilot ring is the gate — a bad rollout is caught before it reaches production.*

2. Requirements Guide — Enterprise App Catalog Deployment in Microsoft Intune

Verify **every** requirement before deploying a catalog app. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-intune-solutions.md`.

Throughout this solution we use a **worked example** so every rule is concrete: a common Windows application we refer to as `Sample App`, selected from the Enterprise App Catalog and deployed to a fictitious tenant `contoso.com`. Replace `Sample App` with the actual catalog app you choose. *Whether any specific application is in the catalog is defined by Microsoft — always confirm your app is present (verify against current Microsoft documentation).*

2.1 Licensing

Description: An Intune-bearing license plan (e.g. Intune Plan 1). App deployment is a built-in Intune capability. **Why:** Intune stores/serves the app content, runs the IME, and reports install state; the Enterprise App Catalog is delivered through the Intune app-management service. **Verify:** M365 admin center → **Billing** → **Licenses**; Intune shows as active. **Implement:** Assign M365 E3/E5 or Business Premium (Intune included), or standalone Intune Plan 1. *Whether the Enterprise App Catalog requires any additional licensing entitlement is Microsoft-defined — verify against current Microsoft documentation for your plan and national cloud.*

2.2 Device enrollment and the Intune Management Extension (IME)

Description: Targeted Windows devices enrolled in Intune, with the IME present and healthy. **Why:** Catalog (Win32) apps are installed only by the IME on enrolled Windows devices. **Verify:** Intune → **Devices** → **All devices** lists the device; run `ENTAPPCAT-Test-Install.ps1` on a device (it checks for the `Microsoft Intune Management Extension` service). **Implement:** Enable automatic enrollment and enroll the device. The IME installs automatically once a Win32/catalog app or PowerShell script is assigned to the device.

2.3 The app is present in the Enterprise App Catalog

Description: The application you want to deploy is a **curated, prepackaged** app in the catalog. **Why:** Only catalog apps get Microsoft's prebuilt package + detection + update management. If the app is not in the catalog, you must use hand-packaged Win32 or the winget/Store lane instead. **Verify:** Intune → **Apps** → **Add** → app type **Enterprise App Catalog app** → **search/browse** for `Sample App` (exact menu labels *verify against current Microsoft documentation*). Confirm it appears. **Implement:** If the app is absent, choose another lane: package it as a Win32 `.intunewin`, or use the **"Winget & Microsoft Store App Management with Microsoft Intune"** product for the winget/Store route. Do **not** assume an app is in the catalog — the catalog's contents change over time.

2.4 Install context decision (system vs user)

Description: Whether the app installs per-device (**system** context) or per-user (**user** context). The catalog package exposes the supported install behavior. **Why:** Most line-of-business apps install in **system** context; the wrong context causes per-user reporting confusion or install failures. **Verify:** Review the install-behavior setting the catalog package presents when you add `Sample App` (the available options are Microsoft-defined per app — *verify against current documentation*). **Implement:** Choose **System** unless the app must be per-user. Record the choice — you reuse it in testing.

2.5 Update management understood and decided

Description: A decision on whether to enable **update management** for the catalog app, and an understanding of how updates are applied. **Why:** Update management is the catalog's headline benefit (apps stay current without re-wrapping), but the **cadence, version control, and exact behavior are Microsoft-defined** and must be understood before you rely on them. **Verify:** Review the update-management option presented when configuring `Sample App` (the toggle name and behavior *verify against current Microsoft documentation*). **Implement:** Decide to enable update management (recommended for staying current) or pin a version; document the choice. Do **not** promise a specific update SLA the service does not guarantee.

2.6 Pilot and broad ring groups

Description: Two Microsoft Entra security groups to ring the rollout — a small pilot and the broad fleet. **Why:** Every deployment is piloted before fleet rollout so a bad rollout is caught early. **Verify:** Entra → **Groups**; the two groups exist (or run `ENTAPPCAT-Set-Assignment.ps1`, which can create them — default `-WhatIf`). **Implement:** Create `Catalog Apps – Pilot Devices – Ring 1` (a few representative devices) and `Catalog Apps – Broad Devices – Ring 2` (the rest of the fleet). Reuse these exact names everywhere.

2.7 Network access and content delivery

Description: Outbound HTTPS from targeted devices to the Intune app-content / CDN endpoints. **Why:** The IME downloads catalog-app content and update content from Microsoft's delivery endpoints. **Verify:** From a device, confirm access to Microsoft's published Intune/Win32 content-delivery endpoints (the exact URL & IP ranges are Microsoft-published — *verify against the current Microsoft list*). **Implement:** Allow Microsoft's published Intune / Delivery Optimization / CDN URL & IP ranges through proxy/firewall. Consider **Delivery Optimization** for large fleets to reduce WAN load.

2.8 National cloud / Government tenant differences (if applicable)

Description: Whether your tenant is Commercial or a US-sovereign cloud (GCC / GCC High / DoD), which can change portal URLs and feature availability. **Why:** Admin portal URLs (e.g. `endpoint.microsoft.us`), Enterprise App Catalog availability, and update-management behavior **may differ or lag** in Government clouds. **Verify:** Confirm your cloud and portal (Commercial `intune.microsoft.com` vs Government `endpoint.microsoft.us`), and confirm catalog availability there. **Implement:** Use the correct portal for your cloud; *verify Enterprise App Catalog availability and behavior in GCC High / DoD against current Microsoft documentation* before relying on it.

Checklist

- Intune licensing assigned (catalog entitlement verified with Microsoft)
- Devices enrolled; IME healthy — `ENTAPPCAT-Test-Install.ps1`
- `Sample App` confirmed present in the Enterprise App Catalog
- Install context decided (System vs User)
- Update-management decision made (behavior verified with Microsoft)
- `Catalog Apps – Pilot Devices – Ring 1` and `Catalog Apps – Broad Devices – Ring 2` created
- Network / content-delivery endpoints reachable
- National-cloud (Government) differences checked if applicable

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation Guide — Enterprise App Catalog Deployment in Microsoft Intune

Follow these steps **in order** in the **Microsoft Intune admin center** (`https://intune.microsoft.com`; Government cloud: `https://endpoint.microsoft.us` — *verify the correct portal for your national cloud against current Microsoft documentation*).

> ***Model:*** *Confirm the app is in the catalog and select it → configure the supplied install behavior and assignment → enable update management → assign to a **pilot** ring → validate install, detection, and an update → expand to **broad**. The pilot ring is the gate — a bad rollout never reaches production.*
>
> ***Navigation note:*** *Enterprise App Catalog menu labels, blade names, and tab order in the Intune admin center change over time. The paths below reflect the current model; **verify each exact label against current Microsoft documentation** before relying on it. The *sequence* (add → select from catalog → install behavior → detection [prebuilt] → assignment → update management → pilot → expand) is stable.*

Named objects (reuse these names verbatim everywhere)

We use the `<Purpose> – <Scope> – <Stage>` naming convention so every object is self-describing and searchable. For the worked example catalog app `Sample App`:

Object	Name to create
Pilot ring group	`Catalog Apps – Pilot Devices – Ring 1`
Broad ring group	`Catalog Apps – Broad Devices – Ring 2`
The catalog app (as added in Intune)	`Catalog App – Sample App – Prod`

(`Sample App` is a placeholder for the actual catalog application you select — do not assume which apps are in the catalog; confirm in Step 3.)

Step 1 — Create the ring groups

1. Go to **Entra** (`https://entra.microsoft.com`) → **Groups** → **New group**.
2. **Group type: Security**. **Group name:** `Catalog Apps – Pilot Devices – Ring 1`. Add a few representative devices (or members). **Create**.
1. Repeat: **New group** → **Security**, name `Catalog Apps – Broad Devices – Ring 2` (the rest of the fleet). **Create**. *(You may instead run `ENTAPPCAT-Set-Assignment.ps1` to create these; it defaults to `-WhatIf`.)*

Step 2 — Start adding an Enterprise App Catalog app

1. In the Intune admin center, go to **Apps** → **All apps** → **Add**.
2. In **Select app type**, under the **Windows** category, choose **Enterprise App Catalog app** (exact label — *verify against current Microsoft documentation*). Click **Select**.

Step 3 — Select the app from the catalog

1. On the catalog picker, **search for** `Sample App` and select it.

*Which applications (and versions) are offered is Microsoft-defined and changes over time — if the app is not listed, it is not in the catalog; use hand-packaged Win32 or the winget/Store lane instead (see the

sibling "Winget & Microsoft Store App Management" product).*

1. If the catalog offers a **version** choice, select the version you intend to pilot (or choose the option that tracks the latest, if you plan to rely on update management). **Select / Next.**

1. The prepackaged app now supplies the **install/uninstall logic** and the **detection rule** — **you do not author these.** Review the **App information** (name, publisher, description). Set the **Name** to `Catalog App – Sample App – Prod` so it is searchable. **Next.**

Step 4 — Confirm install behavior (the supplied package settings)

On the **Program / install-behavior** step (exact tab name — *verify against current documentation*):

1. Confirm the **Install behavior** — **System** (per-device) unless the app must be per-user (see Requirement 2.4). The available options are supplied by the catalog package.

1. Leave the **install/uninstall commands** and **return codes** as supplied by Microsoft's package — do not override them unless you have a documented reason. **Next.**

Step 5 — Verify the built-in detection rule

On the **Detection rules** step:

1. Confirm the rule type is set to **use the prebuilt/built-in detection** supplied by the catalog package (you do **not** manually author an MSI product code / file / registry rule for a catalog app — Microsoft's package provides it). The exact wording of this option is *verify-with-Microsoft*.

1. Do not replace it with a hand-authored rule unless you deliberately intend to; the built-in rule is what makes catalog reporting reliable. **Next.**

Step 6 — Enable update management

On the **update management / supersedence** step (exact label — *verify against current documentation*):

1. **Enable update management** for `Catalog App – Sample App – Prod` so the app is kept current as Microsoft publishes new versions. *The update cadence, whether you can pin a version, and exactly how updates are applied are Microsoft-defined — verify the current behavior and options against Microsoft documentation before relying on a specific guarantee.*

1. If you must stay on a fixed version, choose the **pin/version** option instead (if offered) and record that decision. **Next.**

Step 7 — Assign to the pilot ring

1. On the **Assignments** step, under **Required**, click **Add group** and select `Catalog Apps – Pilot Devices – Ring 1`. *(Choose **Available for enrolled devices** instead if the app should be optional via Company Portal.)*

1. Do **not** assign `Catalog Apps – Broad Devices – Ring 2` yet — the pilot is the gate.
2. **Next** → **Review + create** → **Create.**

Step 8 — Apply and validate on a pilot device

1. On a pilot device, sync policy: **Settings → Accounts → Access work or school → (account) → Info →

Sync**.

1. Run `ENTAPPCAT-Test-Install.ps1` on the device to confirm the IME is healthy and the app's detection state (read-only).

1. Run `ENTAPPCAT-Validate.ps1` to confirm, PASS/FAIL: the app is deployed, assigned to the pilot group, detected as installed, and has update management enabled.

1. Run `ENTAPPCAT-Report.ps1` to confirm Intune reports **Installed** for the pilot devices and to capture install + update status.

1. Complete the **Testing Guide** — including confirming that an **update applies** via update management — before expanding.

Step 9 — Expand to the broad ring

1. Only after the pilot passes the Testing Guide gate, open `Catalog App – Sample App – Prod` →

Properties → **Assignments** → **Edit** → under **Required** → **Add group** → `Catalog Apps – Broad Devices – Ring 2`. **Review + save.**

1. Re-run `ENTAPPCAT-Report.ps1` over the next cycle and chase any **Failed** devices (see Troubleshooting; for deep break/fix use the "Intune App Deployment Troubleshooting & Failure Remediation" product).

Order of operations

`Ring groups → Add app (Enterprise App Catalog) → Select from catalog → Confirm install behavior → Verify built-in detection → Enable update management → Assign pilot → Validate (install + detection + update) → Expand broad`

5. Testing Guide — Enterprise App Catalog Deployment in Microsoft Intune

Run after the app applies to the pilot ring (Implementation Step 8) and again after expanding to the broad ring. The worked example is `Catalog App – Sample App – Prod`, assigned to `Catalog Apps – Pilot Devices – Ring 1`.

Test 1 — App added from the catalog

Do: Intune → **Apps** → **All apps** → confirm `Catalog App – Sample App – Prod` exists and its type is the **Enterprise App Catalog** app type. **Expected (PASS):** The catalog app is present with Microsoft's prebuilt package attached. **FAIL:** The app is missing or was created as a hand-packaged Win32 app instead — re-add via the Enterprise App Catalog app type (Implementation Step 2).

Test 2 — App reports applicable to the pilot ring

Do: Open `Catalog App – Sample App – Prod` → **Device install status**. **Expected (PASS):** Pilot devices are targeted and **not Not applicable**. **FAIL:** Devices show **Not applicable** — confirm the device is in `Catalog Apps – Pilot Devices – Ring 1` and meets any requirement (see Troubleshooting §B).

Test 3 — Install succeeds on the device

Do: On a pilot device, sync, then run `ENTAPPCAT-Test-Install.ps1`. **Expected (PASS):** The IME service is healthy and the app is present / detected on the device. **FAIL:** IME missing or app absent — sync and re-check (Troubleshooting §C / §F).

Test 4 — Built-in detection agrees with Intune (reporting is true)

Do: Compare Intune → the app → **Device install status** with `ENTAPPCAT-Validate.ps1` on-device. **Expected (PASS):** Intune reports **Installed** and the on-device state agrees — no "installed but failed" mismatch and no reinstall loop. **FAIL:** Mismatch — this is unusual for a catalog app (Microsoft supplies the detection rule); capture the IME log and see Troubleshooting §A before escalating.

Test 5 — Update management is enabled

Do: Open `Catalog App – Sample App – Prod` → the **update management** setting; or run `ENTAPPCAT-Validate.ps1` (it reports update management on/off). **Expected (PASS):** Update management is **enabled** (or the intended version pin is set). **FAIL:** Update management is off unexpectedly — re-enable (Implementation Step 6).

Test 6 — An update applies via update management

Do: When Microsoft publishes a newer version of the catalog app (or on the next update cycle), run `ENTAPPCAT-Report.ps1` and confirm the deployed version advances on pilot devices. **Expected (PASS):** Pilot devices move to the newer version without you re-wrapping or re-uploading a package. **FAIL:** Version does not advance after a cycle — confirm update management is enabled and allow a full policy/update cycle; *update cadence is Microsoft-defined (verify current behavior)* before concluding it is broken (Troubleshooting §D).

Test 7 — Fleet report

Do: Run `ENTAPPCAT-Report.ps1 -CsvPath C:\Temp\entappcat.csv`. **Expected (PASS):** A clear per-device Installed / Failed / Not applicable breakdown (and version where available) across the assigned rings, exported to CSV.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–5 pass on `Catalog Apps – Pilot Devices – Ring 1`; install + detection agree; update management enabled
Update pass	Test 6 passes — an update applies via update management on a pilot device
Broad pass	Tests 3, 4, 7 pass for `Catalog Apps – Broad Devices – Ring 2` after expansion

Do **not** assign the broad ring until the pilot gate passes.

6. Troubleshooting Guide — Enterprise App Catalog Deployment in Microsoft Intune

Use `ENTAPPCAT-Discovery.ps1` to see catalog apps + assignments + update settings, `ENTAPPCAT-Test-Install.ps1` for on-device IME + install state, `ENTAPPCAT-Validate.ps1` for the PASS/FAIL pilot gate, and `ENTAPPCAT-Report.ps1` for fleet status. The Intune Management Extension log on the device is `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\IntuneManagementExtension.log`.

> *These are the **catalog-specific first steps**. For deep, cross-cutting Win32/LOB/Store break/fix, hand off to the existing **"Intune App Deployment Troubleshooting & Failure Remediation (Win32/LOB/Store)"** product.*

A. The app I want isn't in the catalog

Symptom	Cause	Remediation
`Sample App` doesn't appear in the Enterprise App Catalog picker	The app is not curated by Microsoft, or catalog contents changed	Confirm the exact name; the catalog's contents are Microsoft-defined and change over time (*verify against current documentation*). If it's genuinely absent, use another lane: hand-package it as Win32, or use the winget/Store product
Only some versions are offered	Version availability is Microsoft-defined	Select an offered version, or rely on update management to track the latest

B. App shows Not applicable

Symptom	Cause	Remediation
Device never gets the app, shows Not applicable	The device isn't in the assigned group, or a requirement supplied by the package excludes it (architecture / min OS)	Confirm the device is in `Catalog Apps – Pilot Devices – Ring 1`; confirm it meets the package's requirements; re-sync
Whole ring is Not applicable	Assignment targets the wrong group	Re-check the assignment (`ENTAPPCAT-Discovery.ps1`) and re-assign to the correct ring

C. Install fails outright

Symptom	Cause	Remediation
App reports Failed on the device	Content download blocked, or an in-use file / prerequisite issue	Confirm network access to Microsoft's content-delivery endpoints (Requirement 2.7); sync; review the IME log for the exit code
Content download / CDN error	Proxy/firewall blocking Intune / Delivery Optimization endpoints	Allow Microsoft's published Intune/DO/CDN URL & IP ranges (*verify current list*); retry
Fails only on some devices	Device-specific (disk space, pending reboot, competing installer)	Check the device; re-sync; escalate to the app-troubleshooting product if it persists

D. Update isn't applying (app stays on the old version)

Symptom	Cause	Remediation
Deployed version doesn't advance	Update management not enabled, or the update cycle hasn't run yet	Confirm update management is **enabled** on `Catalog App – Sample App – Prod` (Implementation Step 6 / `ENTAPPCAT-Validate.ps1`); allow a full policy/update cycle
Version pinned intentionally	A version pin was set instead of auto-update	Confirm the intended choice; switch to update management if you want auto-update
Still not advancing after a full cycle	Update cadence/behavior is Microsoft-defined	*Verify the current update-management behavior against Microsoft documentation* before concluding it's broken; then escalate

E. Detection mismatch (installed but shows Failed, or vice-versa)

- Catalog apps use **Microsoft's built-in detection rule**, so this is uncommon. Confirm on-device

with `ENTAPPCAT-Test-Install.ps1` and compare to Intune's **Device install status**. If they disagree, capture the IME log and escalate — do **not** hand-author a detection rule over the catalog app unless Microsoft support directs it.

F. Assignment scope / nothing deploys

Symptom	Cause	Remediation
App assigned but no device installs	Assigned to an empty or wrong group	`ENTAPPCAT-Discovery.ps1` to see the bound groups; confirm `Catalog Apps – Pilot Devices – Ring 1` has members; use `ENTAPPCAT-Set-Assignment.ps1` (preview first) to (re)assign
IME not present	Enrollment/IME issue	The IME installs automatically once a Win32/catalog app or script is assigned; confirm enrollment, sync, and that the `Microsoft Intune Management Extension` service is running (`ENTAPPCAT-Test-Install.ps1`)

G. Government / national-cloud differences

- Enterprise App Catalog availability, portal URLs (e.g. `endpoint.microsoft.us`), and update

behavior can **differ or lag** in GCC / GCC High / DoD. If a step or catalog app behaves differently, confirm you are using the correct portal and *verify catalog availability and behavior for your national cloud against current Microsoft documentation.*

H. Diagnostics quick reference

```
Get-Service -Name 'Microsoft Intune Management Extension'   # IME present/running
Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Uninstall\*',
                 'HKLM:\SOFTWARE\WOW6432Node\Microsoft\Windows\CurrentVersion\Uninstall\*' |
  Select-Object DisplayName, DisplayVersion | Where-Object DisplayName -like '*Sample App*'
```

The IME log (`IntuneManagementExtension.log`) shows the download, install command, exit code, and detection result per app — the first place to look.

I. When to escalate

If the catalog app is present, assigned to the correct ring, the device is enrolled with a healthy IME and synced, network endpoints are reachable, but Intune still reports Failed (or an update won't apply) after a full policy/update cycle, collect the `IntuneManagementExtension.log` and open a Microsoft support case — and use the "Intune App Deployment Troubleshooting & Failure Remediation" product for the deep-dive break/fix workflow.

7. Best Practices — Enterprise App Catalog Deployment in Microsoft Intune

- **Pick the right lane first.** Use the **Enterprise App Catalog** for common apps that are in the catalog; use **hand-packaged Win32** for apps that aren't or that need custom install behavior; use **winget / Microsoft Store** where that source fits (see the sibling "Winget & Microsoft Store App Management" product). Don't hand-package an app that's already curated in the catalog.
- **Assign to a pilot ring first, always.** `Catalog Apps – Pilot Devices – Ring 1` catches a bad rollout before it reaches `Catalog Apps – Broad Devices – Ring 2` — the single most important practice. The pilot is the gate.
- **Trust — but verify — the built-in detection.** Microsoft supplies the detection rule for catalog apps, which is a big reliability win; still confirm on-device with `ENTAPPCAT-Validate.ps1` so reporting is proven, not assumed.
- **Enable update management deliberately.** It keeps catalog apps current without re-wrapping — the headline benefit — but the **cadence and behavior are Microsoft-defined**. Verify the current behavior against Microsoft docs and don't promise an update SLA the service doesn't guarantee.
- **Decide install context up front.** Choose **System** for most line-of-business apps; only use **User** context when the app genuinely requires it, to avoid per-user reporting confusion.
- **Name everything with** `<Purpose> – <Scope> – <Stage>`**.** Self-describing names (`Catalog App – Sample App – Prod`, the ring groups) keep a growing app catalog auditable and searchable, and let scripts reference objects reliably.
- **Validate an update, not just an install.** The catalog's value is staying current — prove an update actually applies on a pilot device (Testing Test 6) before you rely on it fleet-wide.
- **Watch the report and chase laggards.** Track Installed / Failed / Not applicable with `ENTAPPCAT-Report.ps1`; for deep break/fix on a stuck install, hand off to the "Intune App Deployment Troubleshooting & Failure Remediation" product rather than guessing.
- **Check national-cloud differences.** Enterprise App Catalog availability, portal URLs, and update behavior can differ in GCC / GCC High / DoD — verify for your tenant before deploying.
- **Document each app's deployment record.** Record the catalog app name, install context, update management choice (or version pin), and ring assignments for audits and repeatability.

8. Deliverables Summary — Enterprise App Catalog Deployment in Microsoft Intune

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What the Enterprise App Catalog is, the three Windows-app lanes, the Entra "Enterprise applications" disambiguation, benefits, limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Intune Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered Intune steps: add catalog app, select from catalog, install behavior, built-in detection, update management, pilot assignment, expand
5. Testing Guide	Validation procedures with pass/fail gates, including an update-applies test
6. Troubleshooting Guide	Field issues and remediation (catalog-specific first steps)
7. Best Practices	Field-proven recommendations

PowerShell scripts

Script	What it does
`ENTAPPCAT-Discovery.ps1`	Read-only inventory of catalog / Enterprise App Catalog apps deployed, their assignments, and update settings (Microsoft Graph)
`ENTAPPCAT-Validate.ps1`	PASS/FAIL: a pilot catalog app is deployed, assigned, detected, and has update management enabled (Microsoft Graph)
`ENTAPPCAT-Report.ps1`	Install + update status export to CSV across the fleet (Microsoft Graph, read-only)
`ENTAPPCAT-Test-Install.ps1`	Read-only on-device check of IME health and the app's install/detection state
`ENTAPPCAT-Set-Assignment.ps1`	The only state-changer: assigns a catalog app to a pilot group (and can create ring groups); defaults to `-WhatIf`

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Outcome

Common Windows applications deployed from a **curated, prepackaged source** and kept current through **update management** — without hand-packaging every installer or chasing every version — proven on a pilot ring before scaling to the fleet.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

ENTAPPCAT-Discovery.ps1

Purpose: Read-only discovery of the Enterprise App Catalog (prepackaged Win32) apps in the tenant: which catalog apps exist, what groups they are assigned to, and whether update management / auto-update is enabled. Use it to see the current state before you add or expand an app. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ENTAPPCAT-Discovery.ps1 GRAPH SCOPES DeviceManagementApps.Read.All (delegated, read-only) EXPECTED RESULTS Per catalog app: display name, publisher, install context, update-management state, and the list of assigned groups (id + intent). No changes are made. .NOTES Intune Solutions. Read-only against Microsoft Graph. The exact OData type / property names for Enterprise App Catalog apps evolve — verify the current Graph type ('microsoft.graph.win32CatalogApp' at time of writing) and update-management property against current Microsoft documentation. > [CmdletBinding()] param([string]$GraphEnvironment = 'Global' # 'USGov' / 'USGovDoD' for national clouds — verify with Microsoft)

How to run: powershell -ExecutionPolicy Bypass -File .\ENTAPPCAT-Discovery.ps1 GRAPH SCOPES DeviceManagementApps.Read.All (delegated, read-only) EXPECTED RESULTS Per catalog app: display name, publisher, install context, update-management state, and the list of assigned groups (id + intent). No changes are made. .NOTES Intune Solutions. Read-only against Microsoft Graph. The exact OData type / property names for Enterprise App Catalog apps evolve — verify the current Graph type ('microsoft.graph.win32CatalogApp' at time of writing) and update-management property against current Microsoft documentation. > [CmdletBinding()] param([string]$GraphEnvironment = 'Global' # 'USGov' / 'USGovDoD' for national clouds — verify with Microsoft)

ENTAPPCAT-Report.ps1

Purpose: Read-only fleet report of how Enterprise App Catalog (and Win32) apps are deploying and updating — Installed / Failed / Not applicable / Pending counts per app, and (with -CsvPath) a per-device row including the reported version so you can see update progress across rings. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ENTAPPCAT-Report.ps1 powershell -ExecutionPolicy Bypass -File .\ENTAPPCAT-Report.ps1 ` -AppName "Catalog App – Sample App – Prod" -CsvPath C:\Temp\entappcat.csv GRAPH SCOPES DeviceManagementApps.Read.All (delegated, read-only) EXPECTED RESULTS A per-app status summary on screen and, with -CsvPath, a per-device CSV (device, app, install state, version). No changes are made. .NOTES Intune Solutions. Read-only against Microsoft Graph. Enterprise App Catalog Graph type ('win32CatalogApp') is verify-with-Microsoft. > [CmdletBinding()] param([string]$AppName, [string]$CsvPath, [string]$GraphEnvironment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\ENTAPPCAT-Report.ps1 powershell -ExecutionPolicy Bypass -File .\ENTAPPCAT-Report.ps1 ` -AppName "Catalog App – Sample App – Prod" -CsvPath C:\Temp\entappcat.csv GRAPH SCOPES DeviceManagementApps.Read.All (delegated, read-only) EXPECTED RESULTS A per-app status summary on screen and, with -CsvPath, a per-device CSV (device, app, install state, version). No changes are made. .NOTES Intune Solutions. Read-only against Microsoft Graph. Enterprise App Catalog Graph type ('win32CatalogApp') is verify-with-Microsoft. > [CmdletBinding()] param([string]$AppName, [string]$CsvPath, [string]$GraphEnvironment = 'Global')

ENTAPPCAT-Set-Assignment.ps1

Purpose: Creates (or reuses) the pilot Entra security group and assigns an existing Enterprise App Catalog app to it as Required, so you can pilot the app before broad rollout. This is the ONLY script in the set that changes state. DEFAULTS TO -WhatIf: it PREVIEWS the group create + assignment and changes NOTHING until you explicitly pass -WhatIf:$false. Run the preview first, confirm the target, then apply. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\ENTAPPCAT-Set-Assignment.ps1 `-AppName "Catalog App – Sample App – Prod" Apply: powershell -ExecutionPolicy Bypass -File .\ENTAPPCAT-Set-Assignment.ps1` -AppName "Catalog App – Sample App – Prod" -WhatIf:$false GRAPH SCOPES DeviceManagementApps.ReadWrite.All, Group.ReadWrite.All (delegated; write) EXPECTED RESULTS In preview: the group and assignment that WOULD be created. When applied: the pilot group exists and the app is assigned Required to it. Re-check with ENTAPPCAT-Validate.ps1. .NOTES Intune Solutions. Default -WhatIf; supports ShouldProcess. Assigns to the PILOT ring only — expand to the broad ring from the Intune portal after the pilot passes (see the Implementation Guide). Enterprise App Catalog Graph type is verify-with-Microsoft. > [CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')] param([Parameter(Mandatory)] [string]$AppName, [string]$PilotGroupName = 'Catalog Apps – Pilot Devices – Ring 1', [string]$GraphEnvironment = 'Global') Make -WhatIf the default unless the caller explicitly turns it off. if (-not $PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = $true }

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\ENTAPPCAT-Set-Assignment.ps1 `-AppName "Catalog App – Sample App – Prod" Apply: powershell -ExecutionPolicy Bypass -File .\ENTAPPCAT-Set-Assignment.ps1` -AppName "Catalog App – Sample App – Prod" -WhatIf:$false GRAPH SCOPES DeviceManagementApps.ReadWrite.All, Group.ReadWrite.All (delegated; write) EXPECTED RESULTS In preview: the group and assignment that WOULD be created. When applied: the pilot group exists and the app is assigned Required to it. Re-check with ENTAPPCAT-Validate.ps1. .NOTES Intune Solutions. Default -WhatIf; supports ShouldProcess. Assigns to the PILOT ring only — expand to the broad ring from the Intune portal after the pilot passes (see the Implementation Guide). Enterprise App Catalog Graph type is verify-with-Microsoft. > [CmdletBinding(SupportsShouldProcess, ConfirmImpact='High')] param([Parameter(Mandatory)] [string]$AppName, [string]$PilotGroupName = 'Catalog Apps – Pilot Devices – Ring 1', [string]$GraphEnvironment = 'Global') Make -WhatIf the default unless the caller explicitly turns it off. if (-not $PSBoundParameters.ContainsKey('WhatIf')) { $WhatIfPreference = $true }

ENTAPPCAT-Test-Install.ps1

Purpose: Read-only check, run ON a pilot device, that confirms (a) the Intune Management Extension (IME) is present and healthy and (b) the deployed catalog app is actually installed on this device — by matching its display name against installed programs (the built-in detection rule is Microsoft's; this confirms the on-device result). No changes are made. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ENTAPPCAT-Test-Install.ps1 -AppDisplayName "Sample App" GRAPH SCOPES None — runs locally on the device; does not call Microsoft Graph. EXPECTED RESULTS IME service status, and INSTALLED / NOT INSTALLED for the app based on the machine's installed- programs registry. Read-only. .NOTES Intune Solutions. Read-only; no system changes. Run on a targeted pilot device. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$AppDisplayName)

How to run: powershell -ExecutionPolicy Bypass -File .\ENTAPPCAT-Test-Install.ps1 -AppDisplayName "Sample App" GRAPH SCOPES None — runs locally on the device; does not call Microsoft Graph. EXPECTED RESULTS IME service status, and INSTALLED / NOT INSTALLED for the app based on the machine's installed- programs registry. Read-only. .NOTES Intune Solutions. Read-only; no system changes. Run on a targeted pilot device. > [CmdletBinding()] param([Parameter(Mandatory)]

[string]$AppDisplayName)

ENTAPPCAT-Validate.ps1

Purpose: Read-only validation gate for a single Enterprise App Catalog app. Confirms, PASS/FAIL: (1) the catalog app exists in Intune, (2) it is assigned to the pilot group, (3) Intune reports it Installed on at least one device (detection agrees), (4) update management is enabled. Run this before expanding from the pilot ring to the broad ring. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ENTAPPCAT-Validate.ps1 `-AppName "Catalog App – Sample App – Prod"` -PilotGroupName "Catalog Apps – Pilot Devices – Ring 1" GRAPH SCOPES DeviceManagementApps.Read.All, Group.Read.All (delegated, read-only) EXPECTED RESULTS A PASS/FAIL line per check and an overall gate result. No changes are made. .NOTES Intune Solutions. Read-only against Microsoft Graph. Enterprise App Catalog Graph type ('win32CatalogApp') and the update-management property name are verify-with-Microsoft; adjust if Microsoft renames them. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$AppName, [string]$PilotGroupName = 'Catalog Apps – Pilot Devices – Ring 1', [string]$GraphEnvironment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\ENTAPPCAT-Validate.ps1 `-AppName "Catalog App – Sample App – Prod"` -PilotGroupName "Catalog Apps – Pilot Devices – Ring 1" GRAPH SCOPES DeviceManagementApps.Read.All, Group.Read.All (delegated, read-only) EXPECTED RESULTS A PASS/FAIL line per check and an overall gate result. No changes are made. .NOTES Intune Solutions. Read-only against Microsoft Graph. Enterprise App Catalog Graph type ('win32CatalogApp') and the update-management property name are verify-with-Microsoft; adjust if Microsoft renames them. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$AppName, [string]$PilotGroupName = 'Catalog Apps – Pilot Devices – Ring 1', [string]$GraphEnvironment = 'Global')